

um-game-playing-strength-of-mcts-variants

7位解法：Tree + NNアンサンブル

Rank	7
Team	gezi
Author	gezi
Topic ID	549617
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549617>

Retrieved with Kaggle CLI on 2026-07-03.

CV

私の場合、Gameで分けるGKFの方がCVとLBに整合することもあれば、GameRulesetName によるGKFの方が良いこともあり、少し混乱しました。終盤は、テストでは**新しいGameが支配的**だと考えて、GameによるGKFとラベルによる層化に固定しました。しかしPrivate LBを見ると、このCVで決めた最終アンサンブル重みはまったく機能せず、誤った判断だった可能性があります。

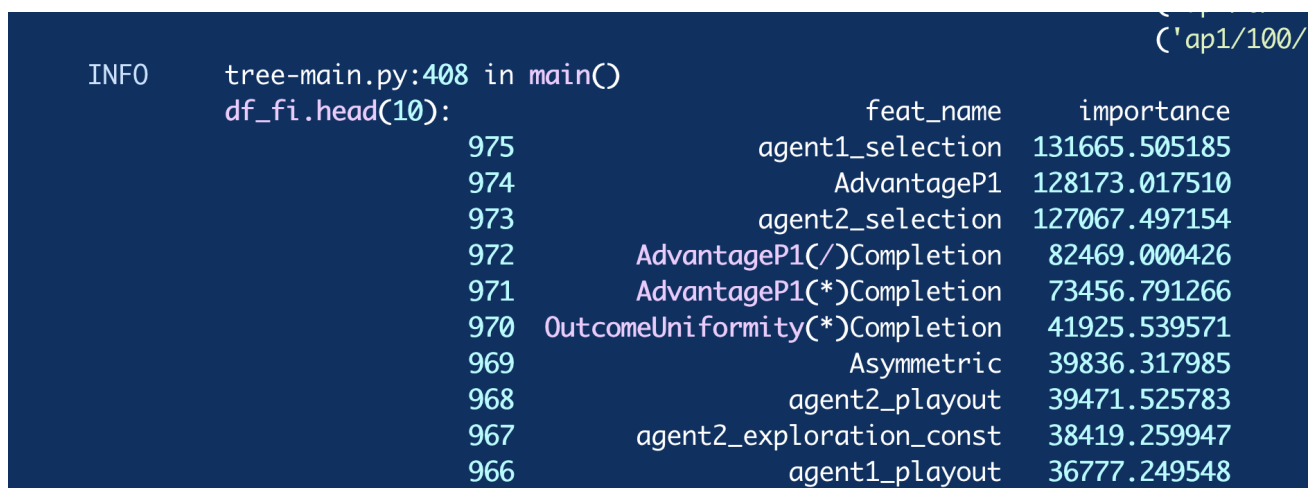
データ拡張

他チームと同じく、かなり早い段階からagent1とagent2、AdvantageP1、ラベルを入れ替えるflip augmentationを使い、LBが約0.01改善しました。反転データを組み合わせるtest-time augmentation (TTA) も使いました。

Treeモデル

私にはTreeモデルがNNより良く、LBでもはるかに安定していました。Treeモデルの性能を引き出す鍵は次のとおりです。

1. 上位20個の数値特徴量について、積・商などの**手作業の交差特徴量**を作る。上位5、10、15、20個と増やすにつれてCV・LBとも大きく改善しましたが、25×25まで増やすとLBが悪化しました。



	feat_name	importance
975	agent1_selection	131665.505185
974	AdvantageP1	128173.017510
973	agent2_selection	127067.497154
972	AdvantageP1(Completion)	82469.000426
971	AdvantageP1(*)Completion	73456.791266
970	OutcomeUniformity(*)Completion	41925.539571
969	Asymmetric	39836.317985
968	agent2_playout	39471.525783
967	agent2_exploration_const	38419.259947
966	agent1_playout	36777.249548

1. Target encodingでLBが0.001改善。
2. 木を深くする。LGBはdepth 16、CatBoostはdepth 10。
3. 過学習を防ぐため、正則化lambdaを10程度、CatBoostの random_strengthも10程度と高くする。
4. LGBのDart modeはCVを改善したもののLBを悪化させました。GameによるGKFが間違っていたのかもしれない。ここではLBを信頼しました。
5. Treeモデルの結果：LGB単体10-foldはPublic 0.419 / Private 0.425。CatBoost単体5-foldはPublic 0.422 / Private 0.428。最終週にCatBoost改善へ多大な努力をしましたが、最終Privateには寄与しませんでした。LGB + CatBoostの単純アンサンブルもPublicを0.001弱改善しただけで、Privateは改善しませんでした。

NNモデル

素晴らしい公開Notebook [MCTS: DeepTables NN](#) を参考にしました。

PyTorchでNNを実装し、上位数値特徴量をbin化し、LGBのleaf特徴量（depth 5、200 trees）を追加しました。pooling layerにはCIN + FMを使い、NNにはlabel augmentation（multinomial）も使いました。

しかしNNはLBで不安定で、CV（GameによるGKF）でも悪い結果でした。運よく5-fold NNの1つがPublic 0.419 / Private 0.427になりました。ランダム性のため通常はPublic 0.423前後でしたが、PublicとPrivateが整合していたため、最良PublicのNNを使えばよいという判断です。

後処理

全体RMSEに対して、OOF予測の $a * x + b$ を最適化しました。

アンサンブル

最良モデルはLGB（10 folds）とNN（5 folds）を重み1対0.75で単純ブレンドし、Public 0.416 / Private 0.423でした。

もっと時間があれば、GameRulesetName によるGKFへ戻し、CVとLBの両方で検証し、OOFを用いたstackingを試します。

補足Q&A

Andreas Bisiadis: 「OOFについて全体RMSEに対して $a * x + b$ を最適化」とはどういう意味ですか。元の予測targetより良くなるのはなぜですか？

gezi: 予測分布を移動させるためです。予測の平均がラベルの平均に近づくことを確認できます。

Vasilis: 「上位20個の数値特徴量を掛け算・割り算した手作業の交差特徴量」とは、重要度上位10個の特徴量同士を乗算・除算するという意味ですか？

gezi: はい、そのとおりです。

stefanocls: 上位数値特徴量の積・商について、相関は確認しましたか？

gezi: いいえ。depth 6のLGBモデルによる上位20特徴量をそのまま使いました。候補選択が本当に重要で、似たdepth 6のCatBoostが出した上位20特徴量では、はるかに悪くなりました。まだ改善余地があります。また、交差特徴量20×20を加えるとLGBがLBでCatBoostより良くなり、これが私にとってLGBの方が良かった理由でもあります。

Vasilis: target encodingとは何を指しますか？

gezi: 各agentのカテゴリ特徴量について、カテゴリ値ごとの `utility_agent1` 平均を特徴量として計算しました。CVは改善しませんでしたでしたがほぼ同じままなので採用し、LBは0.001改善しました。